

Efficient Blockchain Data Processing

End-to-end flow for the Tron blockchain transaction processing system

STEP 1 — Tron Network Connection

Component:

Tron Full Node / TronGrid

What Happens:

- Establish RPC connection.
- Health check node.
- Store last processed block number from DB.

Data Structures:

- `tron_client` RPC object
 - `int last_processed_block_number`
-

STEP 2 — Block Listener (Producer)

Role:

Continuously detects new blocks.

Flow:

1. Fetch current block height.
2. Compare with last processed block.
3. For each new block:
 - Fetch full block data.
 - Push block into `block_queue`.

Data Structures Used:

- `multiprocessing.Queue(maxsize=N)` `block_queue`
- `int` block number
- Optional `LRUCache` for quick retry cache

Why `multiprocessing.Queue`?

Because multiple worker **processes** will consume it.

STEP 3 — Block Queue (Process-Safe Buffer)

Role:

Decouple fetching from filtering.

Data Structure:

```

block_queue = multiprocessing.Queue(maxsize=100)

    (1 Producer)
    Async Block Listener

    multiprocessing.Queue (block_queue)

```

```

Worker 1      Worker 2      Worker 3
(Consumer)    (Consumer)    (Consumer)

```

Behavior:

- Single producer
- Multiple consumers
- FIFO
- Backpressure if full

Production Guarantee:

- No block is lost
- Load-balanced automatically

STEP 4 — Transaction Filter Workers (Parallel Consumers)

Role:

Pick blocks and filter relevant transactions.

Each Worker Does:

1. `block = block_queue.get()`
2. Iterate transactions
3. Apply filtering rules
4. Push relevant transactions to `tx_queue`

Data Structures Used:

Purpose	Structure	Why
Address lookup	<code>set()</code>	O(1) membership
Contract lookup	<code>set()</code> or Trie	Fast match
Block transactions	<code>list</code>	Sequential iteration

Example:

```

addresses = set([...])
contracts = set([...])

```

STEP 5 — Filtered Transaction Queue

Role:

Buffer only relevant transactions.

Data Structure:

```
tx_queue = multiprocessing.Queue(maxsize=1000)
```

OR

```
from queue import PriorityQueue
```

If ordering by block height is required.

Why multiprocessing.Queue?

Multiple processor processes will consume it.

STEP 6 — Transaction Processor Workers

Role:

Consume filtered transactions and process them.

Each Worker:

- 1. tx = tx_queue.get()
- 2. Deduplicate
- 3. Validate
- 4. Enrich
- 5. Store to DB

Data Structures Used:

Purpose	Structure	Why
Deduplication	set()	O(1) check
Fast tx lookup	dict()	txID data
Ordering (optional)	heapq	priority processing

Example:

```
processed_tx_ids = set()
```

STEP 7 — Database Layer

Role:

Persistent storage.

Recommended Structures (DB Level):

Field	Index Type
txID	Hash Index
block_number	B-Tree
address	B-Tree
timestamp	Time-Series Index

Application-Side Temporary Structures:

- dict() cache for recent tx
- Redis cache for hot addresses

STEP 8 — Reorg Handling (Ring Buffer)

Role:

Handle blockchain reorganizations safely.

Data Structure:

```
from collections import deque
recent_blocks = deque(maxlen=20)
```

This acts as a **Ring Buffer / Circular Queue**.

What It Does:

- Stores last 20 blocks
- If reorg happens:
 - Rollback affected tx
 - Remove txIDs from processed set
 - Reprocess new canonical blocks

Important:

Ring buffer is for **cache only**, not main processing.

STEP 9 — Caching & Optimizations

Structures:

Use Case	Structure
Recent tx cache	LRUCache
Sharding by address	dict[address_range queue]
Metrics	Counters / Gauges

Example:

```
from cachetools import LRUCache
recent_tx_cache = LRUCache(maxsize=1000)
```

STEP 10 — Orchestration

Full Flow:

Tron Node
Block Listener (async)
multiprocessing.Queue (block_queue)
Transaction Filter Workers (parallel)
multiprocessing.Queue (tx_queue)
Transaction Processor Workers (parallel)
Database

Data Structures Summary

Step	Component	Recommended Data Structure
1	Tron RPC connection	Object
2	Block Listener	N/A (async loop)
3	Block Queue	multiprocessing.Queue

4	Transaction Filter	HashSet (addresses/contracts)
5	Filtered Transaction Queue	<code>multiprocessing.Queue</code> or <code>PriorityQueue</code>
6	Transaction Processor	Set for txIDs
7	Database	Dict or DB with B-Tree / Time-Series Index
8	Reorg Handling	<code>deque(maxlen=N)</code> (Ring Buffer)
9	Optimizations	LruCache, Sharding structures
10	Orchestration	Combination of above

Addition Layers to make the pipeline process more robust:

- Replace in-memory queue with a true durability engine
- Graceful Shutdown Handling

Replace In-Memory Queue

Instead of:

`multiprocessing.Queue`

Use:

- Kafka
- Redis Streams
- RabbitMQ (durable queue mode)

Why?

Because:

Feature	<code>multiprocessing.Queue</code>	Kafka
Crash safe	No	Yes
Replay capability	No	Yes
Distributed	No	Yes

With Kafka:

- Blocks are persisted to disk
- Consumers track offsets
- Restart resumes automatically

This is what exchanges use.

Graceful Shutdown Handling

On SIGTERM / crash:

1. Stop listener
2. Drain queues
3. Wait for workers to finish
4. Update last processed block
5. Exit cleanly

Example:

```
import signal

def shutdown_handler(signum, frame):
    listener_running = False
    drain_queues()
    update_checkpoint()
    exit(0)

signal.signal(signal.SIGTERM, shutdown_handler)
```

Steps to Implement Master History Table Archiving

1. Identify the Archival Criteria

Define which records should move to history.

Common criteria:

- Blocks older than **N days**
- Blocks **fully processed**
- Blocks **beyond confirmation depth**
- Blocks **less than last_processed_block - safety_margin**

Example:

```
Move blocks where:
block_number < last_processed_block - 10000
AND status = 'PROCESSED'
```

2. Create a History Table

The history table should have **the same schema as the master table**.

Example:

```
CREATE TABLE block_master_history (
    block_number BIGINT,
    block_hash VARCHAR(100),
    parent_hash VARCHAR(100),
    timestamp BIGINT,
    status VARCHAR(20),
    processed_at TIMESTAMP
);
```

Optional improvements:

- Compression
 - Reduced indexing
-

3. Add Indexes to History Table

Indexes help when querying historical data.

Example:

```
CREATE INDEX idx_block_history_blocknum
ON block_master_history(block_number);
```

4. Create an Archival Batch Job

Run a **scheduled job** (cron / airflow / worker).

Example schedule:

Run every day at 2 AM

5. Select Records for Archival

Fetch records eligible for archiving.

Example:

```
SELECT *
FROM block_master
WHERE block_number < :threshold
AND status = 'PROCESSED'
LIMIT 10000;
```

Batching is important to prevent large locks.

6. Insert Records into History Table

Move the selected rows to history.

```
INSERT INTO block_master_history
SELECT *
FROM block_master
WHERE block_number < :threshold
AND status = 'PROCESSED'
LIMIT 10000;
```

7. Delete Archived Records from Master

After successful insert:

```
DELETE FROM block_master
WHERE block_number < :threshold
AND status = 'PROCESSED'
LIMIT 10000;
```

Always delete **only after successful insertion**.

8. Wrap Operations in a Transaction

Ensure atomic operation.

```
BEGIN;
```

```
INSERT INTO block_master_history ...
DELETE FROM block_master ...
```

```
COMMIT;
```

This prevents partial moves.

9. Repeat Until No Records Left

The archival job should loop until all eligible rows are moved.

Example logic:

```
while rows_moved > 0:
    archive_batch()
```

10. Monitor Archival Job

Add monitoring metrics:

- Rows archived
- Job duration

- Errors
- Lag between master and history

Example:

```
archived_blocks_count  
archival_job_latency
```

11. Optional Optimization (Recommended)

Instead of delete, use **partitioning**.

Example:

```
block_master  
  partition_2024  
  partition_2025
```

Then archiving becomes:

```
DETACH PARTITION move to history
```

This is **much faster than row-level delete**.

12. Update Queries to Read From Both Tables

When querying blocks:

```
SELECT * FROM block_master  
UNION ALL  
SELECT * FROM block_master_history
```

Or implement a **view**:

```
CREATE VIEW block_all AS  
SELECT * FROM block_master  
UNION ALL  
SELECT * FROM block_master_history;
```

Final Workflow

Block Processing System

block_master table grows

Scheduled Archival Job

Move old processed blocks

block_master_history

Delete from master

Master table remains small

Benefits

- Faster queries on active blocks
- Reduced index size
- Lower database memory usage
- Improved write performance